

Flaky Test Prediction Without Re-Runs

Towson University

Terrell Beasley, Fallon Katz, Gregory Lawton, Oluwaseyi Salisu, Zaynab Tabassi

{tbeasle1, fkatz3, glawton1, osalisu1, ztabass2}@students.towson.edu

May 19, 2026

Keywords – Flaky Testing, Machine Learning, Random Decision Forest, Flaky Test Prediction, XGBoost, LightGBM

Description:

The purpose of this project is to develop and train a machine learning tool to predict flaky testing in software without implementing reruns. We will be using random decision forests and gradient-boosted models trained on the preexisting FlakeFlagger and iDFlakies datasets to evaluate if feature based models can be used to accurately predict flaky tests.

Assignment of Responsibilities:

Team Members	Roles and Responsibilities
Terrell Beasley	Research Software Engineer
Fallon Katz	Research Software Engineer
Gregory Lawton	Research Software Engineer
Oluwaseyi Salisu	Research Software Engineer
Zaynab Tabassi	Research Software Engineer

Total Budget:

There are no monetary costs associated with this project; however, we can evaluate our budget based on the time that is spent on research, development, and resources used. We have a deadline of approximately two months to complete our project and therefore would have to budget our time across that span.

Deliverables:

Spring 2026

- Initial Proposal
- Progress Report and Progress Report Presentation
- Final Report and Final Report Presentation
- Flaky Predictor
- Feature Extraction Pipeline
- Cross-Project Evaluation

Executive Summary

Title: *Flaky Test Predictions Without Re-Runs*

Date: *May 19, 2026*

By: *Terrell Beasley, Fallon Katz, Gregory Lawton, Oluwaseyi Salisu, Zaynab Tabassi*

Affiliation: *Towson University*

Keywords: *Flaky Testing, Machine Learning, Random Decision Forest, Flaky Test Prediction, XGBoost, LightGBM*

Testing plays an important role in software development today by verifying the quality of code. However, the effectiveness of testing environments is dependent on the quality of the tests that are run, known as *flaky tests*, which can pass or fail without any changes being made to either the code or testing environment. These issues can result in a waste of valuable computational resources by increasing the amount of time it takes to debug problems. Currently, ways that developers identify flakiness in test suites is through rerunning tests multiple times, which can lead to an inefficient use of resources for complex projects. Therefore, a resource efficient way to identify flaky tests during the early development phase of a project is a high necessity.

This project built a machine learning pipeline that predicts whether a Java test is likely to be flaky using only static code features extracted directly from test source files. No reruns, no runtime instrumentation, and no execution history required. A total of 46 features were extracted from 12,699 labeled test methods across 19 open-source projects using both regex-based parsing and an AST-based Java parser. Random Forest, XGBoost, and LightGBM classifiers were trained and evaluated using stratified 5-fold cross-validation, and the approach was compared against smell-only and history-only baselines drawn from the FlakeFlagger dataset.

The project's best model, LightGBM with SMOTE oversampling and threshold optimization, achieved an F1 score of 0.667 on the minority (flaky) class. This outperforms the smell-only baseline by 0.471 F1 and the history-only baseline by 0.187 F1, while sitting only 0.060 below the fully-instrumented FlakeFlagger ceiling of 0.727. A cross-project leave-one-project-out evaluation confirmed that flakiness patterns are highly project-specific, with near-zero cross-project generalization. This is consistent with findings from Luo et al. [3] and Lam et al. [2], who observed that flakiness causes vary significantly across projects, making cross-project generalization a known open challenge in this research area. SHAP analysis identified lines of code, cyclomatic complexity, async wait patterns, and import diversity as the strongest static predictors of flakiness.

Table of Contents

Flaky Test Prediction Without Re-Runs.....	1
Executive Summary.....	2
Table of Contents.....	3
Introduction.....	3
Previous Work.....	4
Methods and Procedures.....	7
Limitations.....	9
Delimitations.....	10
Findings.....	10
Issues.....	12
Conclusions.....	12
Future Work.....	13
Bibliography.....	14
Biographical Sketches.....	14
Task Table.....	16

Introduction

Problem Statement

Flaky tests make software testing less reliable because they can pass or fail even when nothing has changed in the code or its environment. This makes it harder for developers to know whether a failure is caused by a real defect or just inconsistent test behavior. Since flaky tests are often identified by rerunning them over and over again, they can waste both time and computing resources, especially in larger projects. Because of this, there is a need for a more efficient way to identify flaky tests earlier in development without depending so heavily on expensive reruns.

Purpose Statement

The purpose of this project is to develop a machine learning based approach for predicting whether a test is likely to be flaky without relying on repeated reruns. Our project first builds a baseline using existing datasets and models, and then moves toward extracting static and lightweight features directly from Java test code. By doing this, we hope to make flaky test prediction more efficient and more useful earlier in the software development process.

Motivation:

Our group is developing a machine learning tool to predict flaky tests in the software development process. The flaky testing process typically yields passing and failing test results without a change being made to the code or the environment that the code is run in. For developers, flaky testing costs both time and money. For society, improper testing can result in failures that result in anything from a minor inconvenience to a major controversial outcome. Using machine learning to predict where flaky tests may arise reduces the amount of overhead that comes with development and testing; however, a major hurdle that we may have to

overcome is training our machine learning model to properly detect where these flaky tests may arise. Random decision forests and gradient-boosted models may be able to detect flaky tests, but there is also a margin of error where they may detect true defects. To mitigate the impact that this may have on our outcome, we will be testing our machine learning tool on multiple datasets. This will not remove the possibility that true defects will be detected as flaky tests; however, it can be used to improve the certainty that what the machine learning tool is detecting is indeed a flaky test.

Previous Work

Flaky tests are a known issue when it comes to software testing. The same code can be run multiple times without changing any aspect of it, and it may pass or fail inconsistently. With this unpredictable behavior being present, developer confidence is reduced while extra time, money, and energy is spent trying to debug code that seemingly has no problems. As a result, researchers have studied different approaches to flaky tests in order to help detect and prevent these skewed results. The following peer reviewed papers explores several studies relating to flaky tests, their behavior, and techniques for detecting or predicting their presence in software systems.

[1] Abdullah Alshammari et al., “FlakeFlagger: Predicting Flakiness Without Rerunning Tests,” ICSE 2021, DOI: 10.1109/ICSE43902.2021.00140 (repo: <https://github.com/AlshammariA/FlakeFlagger>).

Alshammari et al., approaches the issue of flaky tests by using machine learning to identify where they may appear without having to rerun the tests multiple times [1]. Traditionally, flaky tests are approached by rerunning tests multiple times to observe nondeterministic behaviors in the code. This approach can become expensive, especially when it comes to working with larger programs. The authors’ proposed solution was to construct the dataset built from 24 open-source projects that they would train their machine learning tool on; this tool, called FlakeFlagger, identifies features and methods in the code provided to it to estimate where flaky tests could arise[1].

FlakeFlagger has a few limitations: one of these limitations is that it focuses on runtime and coverage-based features; while this works within the context of their study, it does not provide much use to other projects that may use it because it requires access to build logs, execution traces, or instrumentation, which may not be readily available to developers.

Our aims seek to work on a model that improves upon this by extracting static code features from program test suites, these include, but are not limited to, concurrency, API usage, and code complexity. We will also be evaluating cross-project generalization by training models on tests from one set of projects and evaluating them on different projects; this will be done to assess the accuracy of flaky test detection across different environments. Finally, we will be including feature interpretability analysis using SHAP values, which will allow us to identify the properties within our test code that are most likely to be associated with flaky tests.

[2] Wing Lam et al., “iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests,” ICST 2019 (PDF).

Wing Lam et al., present iDFlakies, a framework to detect and partially classify flaky tests, or tests that exhibit nondeterministic outcomes without changes to the underlying code. The iDFlakies framework detects flaky behavior by repeatedly executing test suites under different execution orders in order to expose inconsistencies caused by test dependencies [2]. The authors applied this framework to a large set of open-source Java projects using Maven and JUnit and discovered 422 flaky tests across 82 projects, demonstrating that flaky behavior is common in real-world software systems [2]. They analyze that flaky tests are almost evenly split between order-dependent and non-order-dependent types, indicating that while test order dependencies are a major cause, other factors also cause flakiness. It was also found that the probability of failure varies widely among flaky tests, making them difficult to detect through simple reruns.

The study contributes a scalable detection framework, an empirical dataset of flaky tests, and practical insights showing that randomized test execution orders are particularly effective for identifying flaky behavior. However, the study also has several limitations. Many of the initially collected projects could not be analyzed due to build failures or configuration issues [2], which significantly reduced the final dataset and may potentially introduce selection bias. Additionally, the framework primarily distinguishes between order-dependent and non-order-dependent flaky tests but does not deeply analyze the underlying root causes of flakiness. Despite these limitations, the dataset and empirical findings presented in the paper provide a valuable foundation for further research. The challenges and costs associated with rerun-based detection methods emphasize the need for approaches that can predict flaky tests without repeated executions.

[3] Q. Luo, et al., "An Empirical Analysis of Flaky Tests," FSE 2014: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, Nov. 2014, pp. 643-653, <https://doi.org/10.1145/2635868.2635920>

Luo, et al., conducted an empirical analysis of flaky tests in order to better understand their causes and characteristics [3]. The researchers examined 201 commits across 51 open-source projects in which developers fixed flaky tests. To organize their findings, they created categories of the primary causes of test flakiness. They found that flakiness is most commonly caused by asynchronous waiting issues, concurrency problems, and test order dependencies. These findings suggest flakiness is often not caused by defects in the code itself, but rather the timing and environmental conditions surrounding the code makes them difficult to detect. This research provides insight into the root causes of flakiness and how and why it might show up rather than predicting it. Our project will build upon this work by applying machine learning to help predict flaky tests without requiring repeated test executions.

[4] Martin Gruber et al., "Practical Flaky Test Prediction using Common Code Evolution and Test History Data," ICST 2023, DOI: 10.1109/ICST57152.2023.00028 (replication package: <https://doi.org/10.6084/m9.figshare.21363075>)

Gruber et al. [4] proposed a flaky test prediction approach that relies entirely on information already available in most software projects, specifically test outcome histories from

the CI system and code churn data from Git. Their key insight was that the weighted flip rate, which measures how often a test switches between passing and failing with more weight given to recent runs, is a strong indicator of flakiness. When combined with code evolution features such as the number of changed files in the current pull request and long term source file change counts, their Gradient Boosting model achieved a 95.5% F1 score using only three features. This was a significant improvement over their industry baseline at BMW, which used a simple heuristic of checking whether a test had five consecutive failures and only reached 67.3% F1. A major strength of their work is its practicality. Unlike FlakeFlagger which requires instrumenting the test runner to measure properties like threading behavior and memory usage, Gruber et al. only needs what Git and a CI system already recorded. This makes their approach easy to deploy in real companies without modifying the testing infrastructure.

However, their approach has a critical limitation: it cannot predict flakiness for newly added tests or tests that have little to no execution history, since the flip rate and duration features require a meaningful number of past runs to be useful. The authors also evaluated on a single industrial project, which raises questions about how well their model generalizes across different codebases and programming languages. Our project addresses these gaps by focusing on static and lightweight dynamic features extracted directly from the test code itself. Because our features are derived from analyzing the code rather than observing past behavior, our approach can classify a test as potentially flaky the moment it is written, before it has ever been executed. This makes our method complementary to Gruber et al.'s history based technique and potentially more valuable in early stage development or for projects that lack extensive CI history.

[5] N. Levin, et al., "Takuan: Using Dynamic Invariants to Debug Order-Dependent Flaky Tests," *2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, Ottawa, ON, Canada, 2025, pp. 81-85, doi: 10.1109/ICSE-NIER66352.2025.00022.

Levin et al. introduce Takuan, a tool designed to help diagnose order-dependent flaky tests, where a test's outcome changes depending on the order in which tests are executed. Such flakiness often occurs when an earlier test modifies a shared state that a later test implicitly depends on [5]. To locate the source of these failures, Takuan collects dynamic invariants from two executions. One execution runs the victim test alone where it passes, while the other runs the polluter before the victim where the test fails. Using the Daikon invariant detection tool, the system records properties about program state. Takuan then compares the invariants from the passing and failing runs to identify problem invariants that appear only in one of the executions. These differences highlight changes in program state that may explain the flaky behavior. In an evaluation across 13 real world Java projects, Takuan produced valid problem invariants for 6 tests and was able to find cleaners for 5 of them [5].

This paper demonstrates several notable advantages. The use of invariant analysis allows the technique to identify concrete differences in program state that explain why a flaky test fails, which can support more precise debugging and repair. The approach is also capable of detecting both internal pollution and external pollution, allowing it to address a wider range of flaky test scenarios than some previous tools. Despite these contributions, the approach has several limitations. Collecting dynamic invariants produces instrumentation overhead, and compatibility issues with the Daikon tool prevented some projects from being analyzed [5]. The method also

relies on observing both passing and failing executions, meaning it still depends on rerunning tests. Even with these limitations, the paper provides useful insight into how differences in program state can reveal flaky behavior.

Methods and Procedures

Methods

This project employed both constructive and empirical research methods. The constructive component involved designing and implementing a static feature extraction pipeline that parses raw Java test source files and produces structured feature vectors without executing the tests. The empirical component involved systematic experimentation: training multiple classifiers on the extracted features, evaluating them under stratified cross-validation, and comparing results against established baselines using standard information retrieval metrics (precision, recall, F1 score).

Plan Overview

The project was executed in five major phases:

Phase 1: Data acquisition and cleaning. The FlakeFlagger dataset (22,236 labeled Java tests across 24 open-source projects) was obtained and cleaned by removing duplicate entries and standardizing the binary label column. A data cleaning script was implemented to report class balance statistics, exported a cleaned dataset for inspection, and served as the input stage for all future model training.

Phase 2: Baseline pipeline. An initial ML pipeline was built using the pre-extracted features included in the FlakeFlagger dataset, which contain runtime and coverage-derived features. Random Forest and XGBoost classifiers were trained and evaluated using stratified 5-fold cross-validation. This phase established the performance ceiling achievable with full runtime instrumentation ($F1 = 0.727$) and identified methodological issues including improper use of standard KFold (which produced misleading results on the imbalanced dataset) and feature leakage from a row index column, both of which were corrected before reporting results.

Phase 3: Static feature extraction. The raw Java source files for all 24 FlakeFlagger projects were cloned using Git sparse checkout, then developed a two-stage feature extractor. The first stage used regular expressions to extract 46 structural, density, environment, and import-based features from each test file. The second stage used the javalang Python library to parse each file into an Abstract Syntax Tree and extract 22 deeper structural features including cyclomatic complexity, nesting depth, instance and static field counts, lambda density, and reflection usage. After matching source files to labeled test names via a package.classname key strategy, a dataset of 12,699 labeled test methods across 19 of 24 projects was produced (5 projects could not be matched due to version mismatches between the FlakeFlagger study period and current repository).

Phase 4: Model optimization and evaluation. Three classifiers, Random Forest, XGBoost, and LightGBM, were trained under multiple configurations: baseline, SMOTE oversampling,

decision threshold tuning, and RandomizedSearchCV hyperparameter search. All models were evaluated using stratified 5-fold cross-validation with F1 score on the flaky class as the primary metric. A leave-one-project-out (LOPO) cross-project evaluation was also implemented to test generalization, a formal baseline comparison against smell-only and history-only classifiers, and SHAP feature importance analysis to interpret model decisions.

Phase 5: Test Set Ordering Experiment. To investigate whether the order in which projects are introduced during training affects model performance, a cumulative training experiment was implemented. Projects were processed sequentially across four different orderings (alphabetical, reversed, and two alternating sequences). For each ordering, when testing on project P, the model was trained only on projects that appeared before P in that sequence. SMOTE was applied to the training portion of each cumulative fold to address class imbalance. This experiment was run for both XGBoost and Random Forest and evaluated on F1, precision, recall, and accuracy.

Schedule

March 9 (Tasks Complete)	Project Proposal Due
March 11 - 14 (Tasks Complete)	Development of Dataset Cleaner
March 15 - 21	Spring Break
March 22 - 29 (Tasks Complete)	Continue Development of Dataset Cleaner
March 30 (In Progress)	Midsemester Progress Report Progress Presentation
March 31 - April 30 (In Progress)	Development of Static Feature Extractor Begin Training Model
May 1 - 10	Wrap Up Development Compile Final Deliverables
May 19	Final Report Final Presentation

Deliverables

Name	Purpose	Contents	Location
Flaky Predictor	Binary classifier predicting test flakiness	Trained RF, XGBoost, LightGBM models evaluated via stratified CV	src/main.py, src/models.py, src/optimize.py

Feature Extraction Pipeline	Extract 46 static features from raw Java test files	Regex + AST extractor, static_features.csv (12,699 rows)	src/static_feature_extractor.py, src/ast_feature_extractor.py
Baseline Comparison	Compare our approach to smell-only and history-only classifiers	Four-way comparison table with F1, precision, recall	src/baselines.py
Cross-Project Evaluation	Test generalization across unseen projects	LOPO results across 15 projects	experiments/cross_project_tests.py
SHAP Analysis	Interpret which features drive predictions	Bar chart, beeswarm plot, ranked feature CSV	src/shap_analysis.py
Final Report	Document all methods, findings, and conclusions	This document	Submitted
Final Presentation	Present findings to course audience	Slides covering introduction through recommendations	Submitted
Ordering Experiment	Test whether project training affects model predictions	Training across r orderings for RF and XGBoost	src/ordering_experiment.py src/ordering_experiment_rf.py

Limitations

First, 5 of the 24 FlakeFlagger projects produced zero matches between the extracted source files and the labeled dataset. This occurred because the FlakeFlagger study labeled tests from a specific historical commit, while the cloned repositories reflect current HEAD. Class names and package structures in projects like Alluxio, OkHttp, and Orbit have changed significantly. As a result, the matched dataset to 19 projects (12,699 tests) compared to the full FlakeFlagger dataset of 24 projects (22,236 tests).

Secondly, the FlakeFlagger labels themselves were generated by running tests many times and observing nondeterministic failures. Some borderline flaky tests may have been mislabeled as stable if they only fail rarely, introducing noise in the ground truth.

Third, all static features were extracted at the class level, not the individual test method level, meaning tests within the same class share the same feature vector. This may reduce discriminative power for classes with mixed flaky and stable test methods.

Delimitations

The study focused exclusively on Java projects, as the FlakeFlagger dataset only covers Java. The analysis focused on identifying whether a test exhibited flaky behavior, without categorizing the underlying source of the flakiness (such as order-dependent, concurrency, network, etc.). In addition, tests from the 5 unmatched projects were excluded rather than attempting to re-label them from scratch, as doing so was outside the scope of the project timeline.

Findings

Overview

The results demonstrate that static source code features alone are sufficient to predict flaky Java tests at a level approaching fully-instrumented systems. The best-performing model, LightGBM with SMOTE oversampling and threshold optimization, achieved $F1 = 0.667$ on the flaky class, surpassing both the smell-only baseline ($F1 = 0.196$) and the history-only baseline ($F1 = 0.480$) by wide margins, and falling only 0.060 below the FlakeFlagger ceiling of 0.727 that requires full runtime instrumentation. Cross-project generalization remains a hard open problem, with near-zero $F1$ under leave-one-project-out evaluation, confirming that flakiness is highly project-specific and constraining to within-project training is substantially more effective.

Detailed Findings

Finding 1: Static features substantially outperform established smell-only detection. The smell-only baseline, which uses the eight test smell labels provided by FlakeFlagger (assertion roulette, conditional test logic, eager test, fire and forget, indirect testing, mystery guest, resource optimism, and test run war), achieved $F1 = 0.196$. This is barely above random on the minority class and confirms that the simple heuristics used by existing test smell detectors are insufficient for accurate flakiness prediction. This finding was established by training the same classifier (LightGBM with SMOTE) on only the smell columns from `processed_data.csv` and evaluating under stratified 5-fold CV, using `src/baselines.py`.

Finding 2: History-based features perform better than smells but require runtime infrastructure. The history-only baseline, using the eight `hIndexModificationsPerCoveredLine` features from FlakeFlagger, which encode how frequently the code covered by each test has been modified across multiple Git history windows, achieved $F1 = 0.480$. This is a meaningful improvement over smells but still substantially weaker than the static approach. Critically, these features require access to both a CI system (to know which lines a test covers) and Git commit history, making them unavailable for new tests or projects without established CI pipelines. The static approach requires neither.

Finding 3: LightGBM with SMOTE and threshold optimization is the best static-feature model. Across all configurations tested, Random Forest, XGBoost, and LightGBM, with and without SMOTE, with and without threshold optimization, and with hyperparameter tuning via `RandomizedSearchCV`, LightGBM consistently outperformed the alternatives. The final model

achieved $F1 = 0.667$, precision = 0.747, and recall = 0.602. Random Forest achieved higher recall (0.75) when its threshold was tuned, but at the cost of precision (0.31), resulting in lower overall F1. This finding emerged from the systematic benchmarking implemented in `src/optimize.py`.

Finding 4: Class imbalance handling is critical to obtaining valid results. Early in the project, using standard KFold cross-validation produced misleading F1 scores ranging from 0.0 to 0.49 because individual folds contained no flaky test examples. Switching to StratifiedKFold, which preserves the flaky/non-flaky ratio in each fold, and applying SMOTE oversampling inside training folds (never applied to the test fold) corrected this. Threshold tuning: sweeping the decision cutoff from 0.01 to 0.99 and selecting the value that maximizes F1 provided additional improvement, particularly for Random Forest where the default 0.5 threshold heavily biases predictions toward the majority class.

Finding 5: Static features cannot fully close the gap to runtime instrumentation. Despite testing 46 static features, AST-derived structural features, SMOTE, threshold tuning, and three classifiers, the performance ceiling with static features alone is approximately $F1 = 0.668$. The gap to the fully-instrumented baseline ($F1 = 0.727$) persists because roughly 40% of flaky tests in the dataset are statistically indistinguishable from stable tests based on any extractable static property. They fail due to race conditions, environmental dependencies, or external service behavior that cannot be seen in the source file at rest.

Finding 6: Flakiness patterns do not generalize across projects. The leave-one-project-out evaluation, implemented in `experiments/cross_project_tests.py`, showed near-zero average F1 across 15 tested projects (RF: 0.001, XGBoost: 0.051). Isolated projects such as `wro4j` ($F1 = 0.444$) and `logback` ($F1 = 0.133$) showed some signal, but most projects produced zero correct flaky predictions when the model was trained on all other projects. This confirms that flakiness is driven by project-specific patterns. The concurrency idioms, external dependencies, and test design choices that make a test flaky in one codebase do not predict flakiness in another.

Finding 7: Project Training Order and cumulative data does affect model performance.

In order to find if there is any impact to the results due to the introduction of a set of projects at different times during training, a test we called "Shuffle Branch" was created and consisted of a cumulative training exercise. The idea was to create an environment similar to continuous online learning. The algorithms in this field (Random Forest and XGBoost) employed 46 source code attributes like number of code lines, concurrency primitives, asynchronous waiting techniques, and import diversity that were used in the main study. To set up this experiment, the projects were sorted using different predetermined permutations. The permutations included order based on structure (such as alphabetical and reversed), size (order in which large projects come first versus small projects first), and curriculum based on data (for example, sorting the projects in descending order based on the projects' flakiness rate, concurrency, or I/O dependency). The model for each of these permutations, when tested, was only trained on the projects before it within the specified ordering. In spite of the significant class imbalance, to ensure that the experiments were still evaluated correctly, SMOTE was performed only on the training set of

each cumulative fold. The results of the test showed that the order does have a considerable effect on the F1 macro-score of the algorithm. The more projects that were tested before the particular project, the higher the F1 score of the algorithm, as the model had been trained to handle larger volumes of training data. Thus, we can conclude that the models require an immense amount of data in order to show the best prediction results.

Issues

Issue 1: *Label Uncertainty*

An issue encountered during this project is label uncertainty in the ground truth dataset. A test may fail due to a real defect in the code but appear flaky if the failure is intermittent. The FlakeFlagger labels were generated by running each test many times and observing nondeterministic failures, which is the most reliable known method for labeling, but borderline tests that fail only rarely may still be mislabeled as stable. This introduces noise into the training data that no ML technique can recover from. This issue was mitigated by relying on the FlakeFlagger dataset, which used a large number of reruns per test to maximize label reliability, rather than attempting independent labeling.

Issue 2: *Data Imbalance*

The dataset is severely imbalanced. Only about 2.12% of the 12,699 tests in the extracted dataset are labeled flaky. A naive classifier that predicts "not flaky" for every test would achieve 97.88% accuracy, making accuracy a meaningless metric. This was addressed through three mechanisms: stratified K-fold cross-validation to preserve the flaky/non-flaky ratio in every fold, SMOTE oversampling applied within each training fold to synthesize additional minority class samples, and decision threshold tuning to find the cutoff that maximizes F1 rather than defaulting to 0.5.

Issue 3: *Project Version Mismatch*

Five of the 24 FlakeFlagger projects produced zero matches between the extracted Java source files and the labeled dataset. The FlakeFlagger authors labeled tests at a specific historical commit, while cloning the repositories today retrieves the current HEAD, in projects like Alluxio, OkHttp, Orbit, elastic-job-lite, and httpcore, package structures and class names have changed significantly enough that the matching strategy (package.classname prefix lookup) finds no correspondence. This was accepted as a dataset limitation and proceeded with 19 of 24 projects (12,699 of 22,236 labeled tests).

Conclusions

This project set out to answer a practical question: Can flaky Java tests be predicted accurately using only static source code features, without running the tests at all? The results show the answer is largely yes. A LightGBM classifier trained on 46 features extracted from raw Java

source files achieved $F1 = 0.667$, outperforming both smell only detection ($F1 = 0.196$) and history based prediction ($F1 = 0.480$) by substantial margins, while closing to within 0.060 of the performance achievable with full runtime instrumentation.

This result has a meaningful implication for the field: the most discriminative signals for flakiness — test size, complexity, concurrency primitives, import diversity, and async waiting patterns are visible directly in the source code before a test is ever executed. Developers could integrate a static analyzer of this kind into a code review tool or IDE plugin to flag potentially flaky tests at write time, with no CI infrastructure required.

However, cross-project evaluation reveals a harder problem: flakiness patterns are highly project specific. A model trained on 17 projects and tested on a held out 18th project achieves near zero $F1$, meaning the features that distinguish flaky from stable tests in one codebase do not reliably transfer to another. This limits the practical utility of a single universal model and suggests that per project fine tuning, or continuous retraining as a codebase evolves, would be necessary for real world deployment.

Finally, the persistence of a gap between static only and instrumented performance. Even after exhaustive feature engineering and model optimization, some potential flakiness is still fundamentally invisible in static code. Tests that fail due to race conditions between threads, external service timeouts, or test ordering effects leave no detectable trace in the source file. Closing this gap entirely would require either runtime information or historical execution data, which reintroduces the infrastructure dependency the static approach was designed to avoid.

Future Work

Future research building on this project should consider the following directions:

Per-project model training and continuous retraining. Cross-project evaluation showed that a universal model does not generalize. A more practical deployment would train a lightweight model per project and retrain it incrementally as new tests are added and flakiness labels are collected from CI runs. This hybrid approach, using static features for new tests and history features as they accumulate, would combine the zero-cost initial prediction of the method with the stronger performance of history-based approaches over time.

Flakiness category classification. Rather than predicting binary flaky/non-flaky, a useful extension would classify the type of flakiness (concurrency, order-dependent, network, timing, etc.) as established by Luo et al. [3]. Category-specific predictions would give developers more actionable guidance on how to fix a flagged test.

Expanded and more recent datasets. The FlakeFlagger dataset was collected in 2020–2021. A more recently labeled dataset, or one that spans additional languages (Python, JavaScript), would allow broader evaluation and reduce the risk that this project’s findings reflect patterns specific to a particular era of Java development.

Integration with developer tooling. The static feature extractor developed in this project could be packaged as a Maven plugin, a GitHub Action, or an IDE extension that flags potentially flaky tests inline during development, before they ever reach CI. Evaluating whether such a tool reduces flaky test density in practice would be a valuable real-world validation of the research.

Bibliography

[1] Abdullah Alshammari et al., “FlakeFlagger: Predicting Flakiness Without Rerunning Tests,” ICSE 2021, DOI: 10.1109/ICSE43902.2021.00140 (repo: <https://github.com/AlshammariA/FlakeFlagger>).

[2] Wing Lam et al., “iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests,” ICST 2019 (PDF).

[3] Q. Luo, F. Hariri, et al., “An Empirical Analysis of Flaky Tests,” FSE 2014: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, Nov. 2014, pp. 643-653, <https://doi.org/10.1145/2635868.2635920>

[4] Martin Gruber et al., "Practical Flaky Test Prediction using Common Code Evolution and Test History Data," ICST 2023, DOI: 10.1109/ICST57152.2023.00028 (replication package: <https://doi.org/10.6084/m9.figshare.21363075>)

[5] N. Levin, et al., "Takuan: Using Dynamic Invariants to Debug Order-Dependent Flaky Tests," *2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, Ottawa, ON, Canada, 2025, pp. 81-85, doi: 10.1109/ICSE-NIER66352.2025.00022.

Biographical Sketches

The research team at Towson University is comprised of a group of undergrad Computer Science students with the collective purpose of developing upon the past principles of flaky test predictions through the utilization of Machine Learning capabilities.

A. Terrell Beasley

Terrell Beasley is an undergraduate student working towards a Bachelor of Science in Computer Science at Towson University. He has completed the Requirements Analysis and Modeling course in which he was tasked to work on a group project to develop a machine learning model to assist students with studying for the course; the model was developed to assist students with comprehending the material that was given to them without giving them direct answers to questions.

B. Fallon Katz

Fallon Katz is a senior undergraduate pursuing a Bachelor of Science degree in Computer Science within Towson University's Software Engineering track. She has completed the

Requirements Analysis and Modeling course at Towson University where she worked with a team to develop a website that utilized a local LLM to assist students with studying for the course. She is also in the process of completing a course in Software Quality Assurance and Testing. She is looking forward to working together and collaborating with groups on important issues relating to Machine Learning.

C. Gregory Lawton

Greg Lawton is an undergraduate senior at Towson University focusing on a Software Engineering track. He is working on a Bachelors of Science in Computer Science with an expected graduation in May, Spring 2026. He is currently spending his time as a full time student and working as a Community Center Manager at Towson University, where he has gained experience managing a team, working in a fast paced environment, and developed high proficiency in Microsoft 365 (word, excel, powerpoint, etc). In 2024, Greg gained his CompTia Network + Certification and has continued to work on various projects in a team dynamic (using Rust, Node.js, Java, HTML, etc), with some of his notable projects being “Admin Arsenal” and “StudyAir”. He is excited to bring his team skills and academic knowledge to support this project's goal.

D. Oluwaseyi Salisu

Oluwaseyi Salisu is a Computer Science major at Towson University on the Software Engineering track with an expected graduation of May 2027. He has hands on experience in machine learning and software engineering through his ML/ML Ops internship at LegaC AI where he built and deployed AI agents and maintained ML pipelines, as well as his fellowship at Headstarter AI where he completed multiple real world projects spanning frontend, backend, cloud, and machine learning. His project experience includes building a fullstack Student Task Manager with React and Spring Boot, developing an AI study companion at MorganHacks 2025, and creating a misinformation detection pipeline using embeddings, FAISS, and RAG that analyzed nearly 45,000 news articles. He looks forward to collaborating with his teammates while deepening his understanding of machine learning and gaining experience in academic research methodology.

E. Zaynab Tabassi

Zaynab Tabassi is an undergraduate student pursuing a Bachelor of Science in Computer Science at Towson University with a focus in Software Engineering. She is expected to graduate in December 2026. Her academic interests include software development and collaborative project development, with a growing interest in web programming. Through her coursework, she has gained hands-on experience building and analyzing software systems in team-based environments. She is excited to work with her team and not only apply her skills but to also gain experience while successfully completing this research project.